UNIVERSITY OF CAMBRIDGE

Research Informatics Training

Managing Bioinformatics Software and Pipelines

Reproducible and scalable bioinformatics for researchers (or: how to do bioinformatics like a pro)

Course overview

Recognise and address challenges in setting up bioinformatics environments to achieve scalable and reproducible analysis

Use package managers to install software

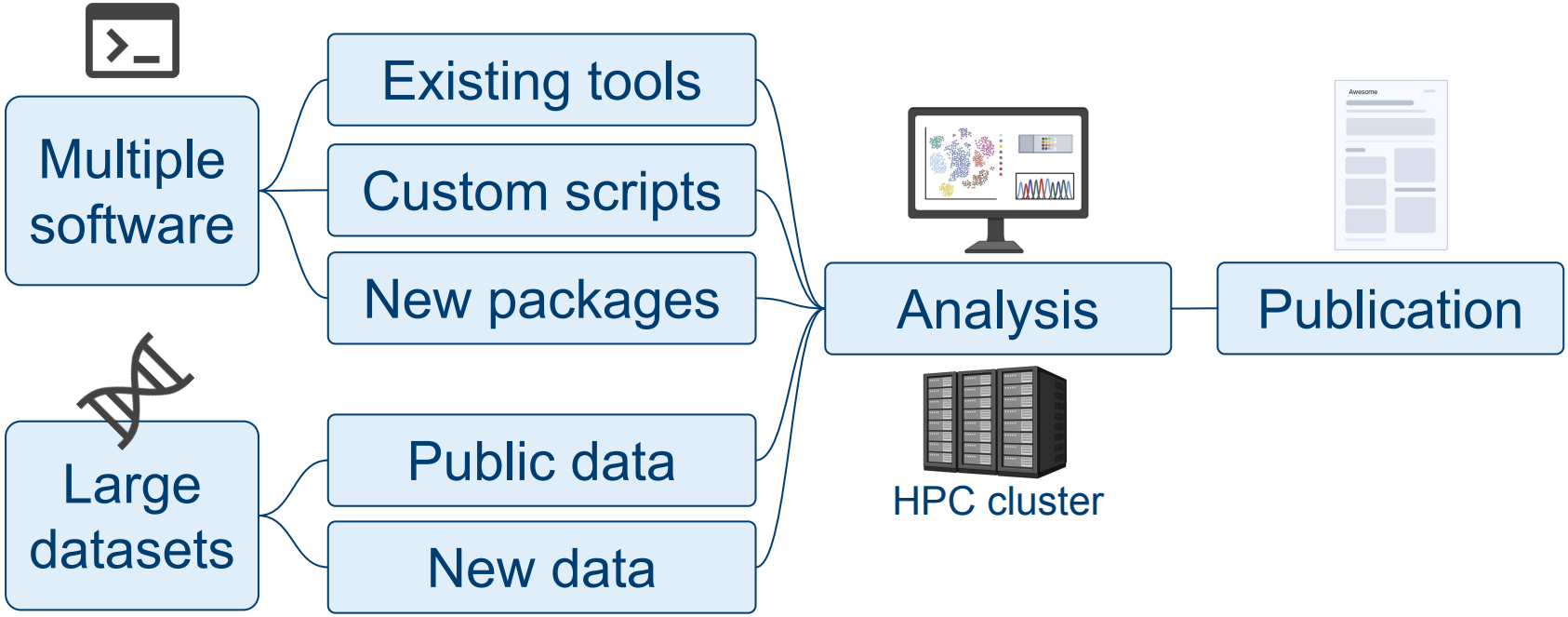
Use software containers for reproducible environments

Hands-on experience with curated Nextflow pipelines

Configuring workflows for HPC clusters

Reproducible and scalable bioinformatics

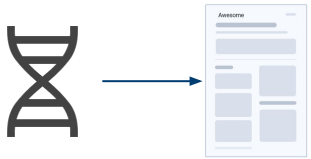
Bioinformatics involves complex data and software



(Pro) Bioinformaticians strive for reproducibility

Ideally we want to trace every result/figure back to the data

Which version of public data did you use?	Which software versions did you use?	Can you describe every command used?



Can I reproduce your analysis?

A decade ago reproducibility was challenging

A real-life example of a PhD student circa 2005…

- Custom Perl scripts to define workflows.
- Each group member has their own Perl script

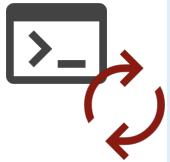
  to split large BLAST jobs and run them on the HPC.
- Hard-coded scripts for the queuing system on that HPC
  - Would not work on a different HPC cluster.
- No software version control.

Reproducibility is made easier by current tools

The same PhD student nowadays:

- Nextflow to define workflows.
- Software versions controlled using package managers (Mamba) and software containers (Singularity).
- Code documented with RMarkdown and/or Jupyter notebooks.
  - Custom code under version control via Git.

Managing software for reproducibility: challenges



Rapid evolution of bioinformatic tools

Compatibility issues across software versions

Handling dependencies and environment variables

Inconsistent software setups that are **hard to reproduce**

Managing software for reproducibility: solutions

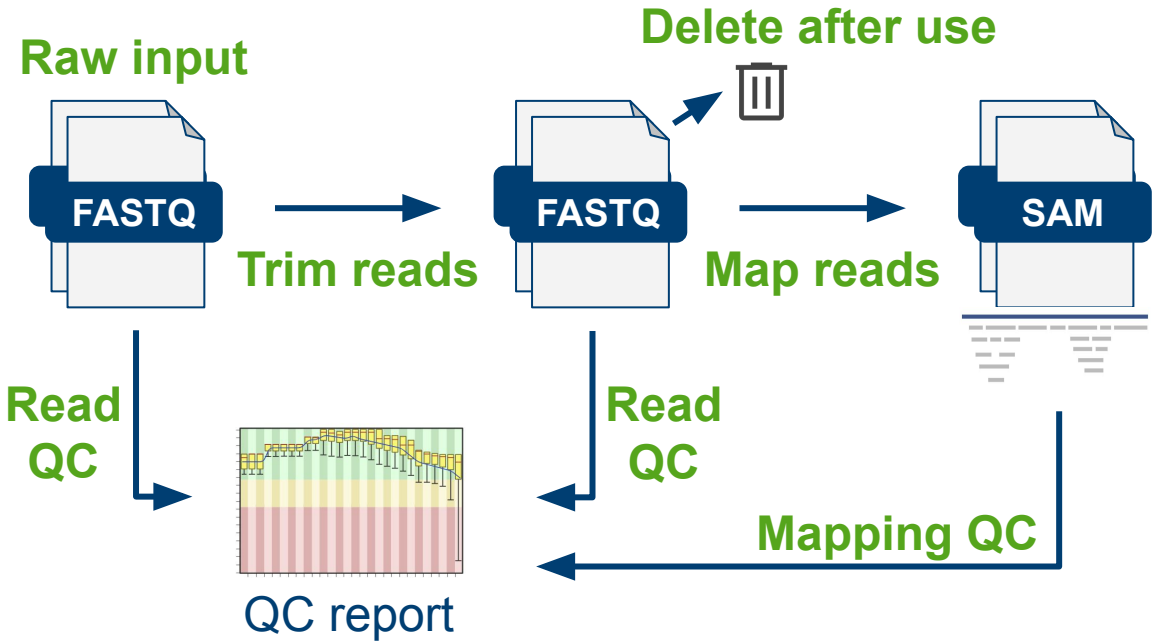
Package managers

Provide “recipes” to install software and all its dependencies.
Software can be partitioned in individual folders, keeping working environments clean.

Software containers

Pre-packaged software within a minimal “virtual computer”.
All dependencies come pre-installed within it.

Managing complex analysis workflows: challenges



We need definitions for:

The relationships between steps of the analysis.

Input and output files in compatible formats.

Managing intermediate files.

Managing complex analysis workflows: solutions

Workflows are formal, executable definitions of how data flows from one processing step to the next.

Nextflow and Snakemake are two popular solutions in bioinformatics.

Step: Trim
Input: FASTQ
Output: FASTQ
Depends: Raw input

Step: Map
Input: FASTQ
Output: BAM
Depends: Trim

Step: quality report
Input: FASTQ & BAM
Output: HTML
Depends: Trim & Map

Use curated and well tested community workflows

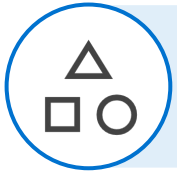
nf-core

Standardised, community-developed pipelines for common analysis

79 released workflows + 37 under development (Mar 2025)

Save time & reduce errors

Can work on your local computer, HPC or cloud



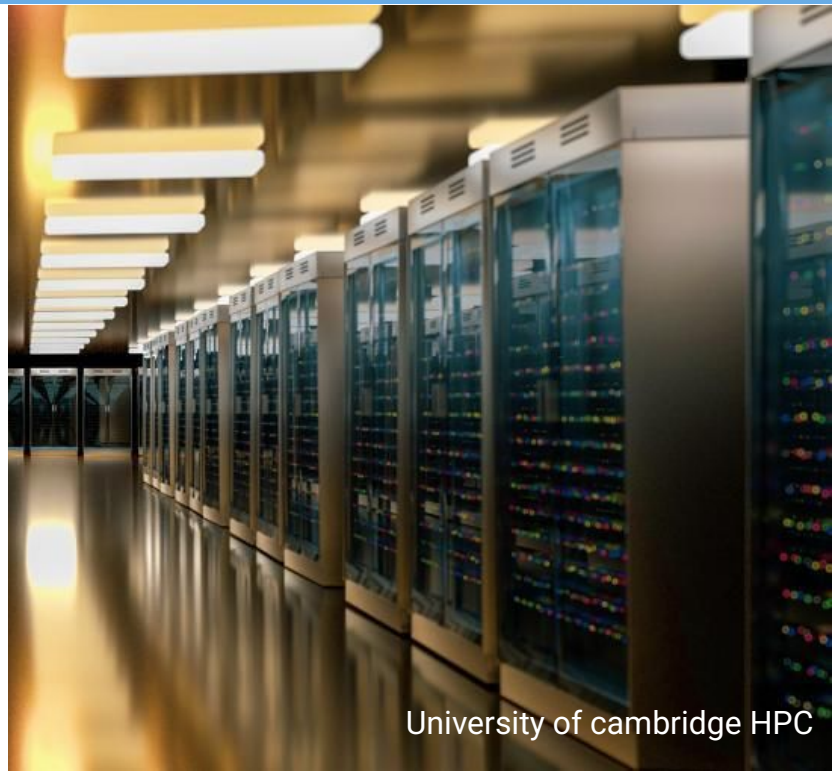
Modular: can be reused in custom pipelines

Continuous development, bug fixes and updates

Scalable solutions are essential for large datasets

High Performance Computing clusters are crucial in fields like genomics.

Workflow systems (e.g. Nextflow) can run seamlessly on HPC clusters (SLURM, LSF, cloud, as well as just locally).



University of cambridge HPC

Reference genome versions: challenges

A common source of confusion in genomic analysis is the **version of the reference genome and annotation**

GENCODE	1	2	3	...	X	Y	MT
Ensembl	chr1	chr2	chr3	...	chrX	chrY	chrM

Same human genome, two naming conventions.

Official genome is called GRCh38, but UCSC and others call it hg38.

Confused? So are we!

Reference genome versions: challenges

- Differing naming conventions.
- No single source of “best” reference for all species.
- No standard descriptions for the state of different references.
- Regular updates in some species (e.g. new gene annotations).
- Most up-to-date version is not always the best, if you want to compare with previous published work (or make sure to run a “liftover”).

Reference genome versions: challenges

Different types of genome FASTA often available

- **Repeat masking**:
  - Unmasked
  - Hard-masked - repeats replaced by ‘N’
  - Soft-masked - repeats in lowercase
- **Assembly type**:
  - Primary (chromosomes and scaffolds)
  - Complete (plus haplotypes and patches)

Reference genome versions: solutions

Think about what is best-suited for your project, what your group uses, what your community uses, what similar publications use.

When in doubt we recommend
primary soft-masked genomes from Ensembl

ensembl.org for vertebrates
ensemblgenomes.org for bacteria, protists, fungi, plants and metazoa

Be prepared to re-run your analysis on a different reference

↓

One more reason for reproducible and reusable workflows!

Don’t feel like you need these solutions all the time!

Exploratory stage	Lots of “hacking about” at the start of a project *Ad hoc* scripts Messier file organisation
Final stage	Well-defined software environments Use of standard workflows Logical file organisation

Don’t feel like you need these solutions all the time!

Not all your work will be best represented as a formal workflow

- Use standard workflows for key data processing steps
- Downstream custom analysis leading to figures may use standard scripts (R, Python, Bash, etc.) or mixed documents such as RMarkdown and Jupyter notebooks

Summary

Use existing tools to help you achieve

reproducible and scalable bioinformatics

- **Package managers** and **software containers** ensure easier and reproducible software installation.
- **Existing workflows** save time, reduce errors and scale analyses to large data and parallel processing on HPC clusters.
- Always **report the versions of public data** being used (e.g. reference genomes and databases).

Package managers

How to use Mamba to manage your software

Managing bioinformatics software: challenges

- Building software from source is an advanced skill if you're not used to administer a Linux server
- Bioinformatics software gets updated regularly:
  - Software dependency conflicts
  - Balancing upgrade vs stability
  - May need different versions in different projects

Software dependencies can be very complex

At first glance Pandas has few dependencies

Required dependencies

pandas requires the following dependencies.

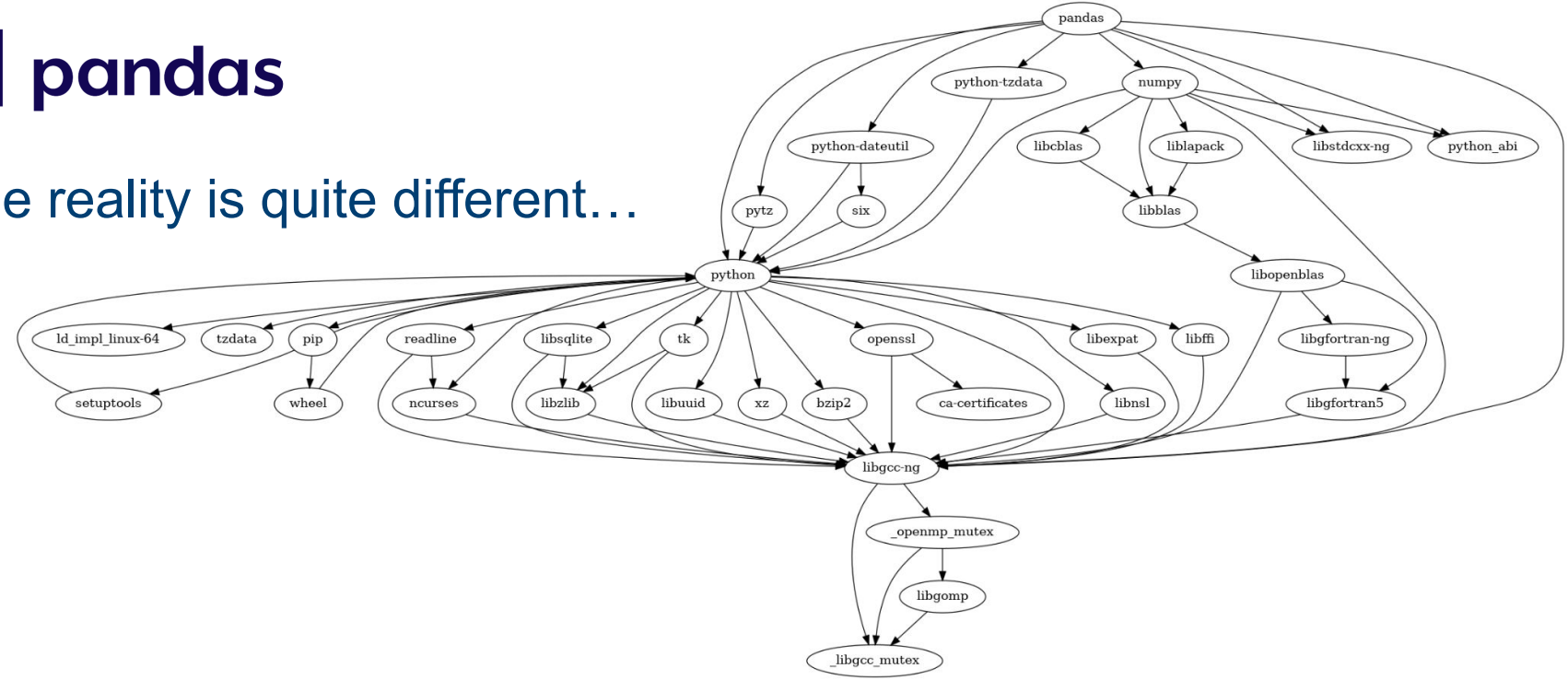
Package	Minimum supported version
NumPy	1.22.4
python-dateutil	2.8.2
pytz	2020.1
tzdata	2022.7

Snapshot from Pandas documentation

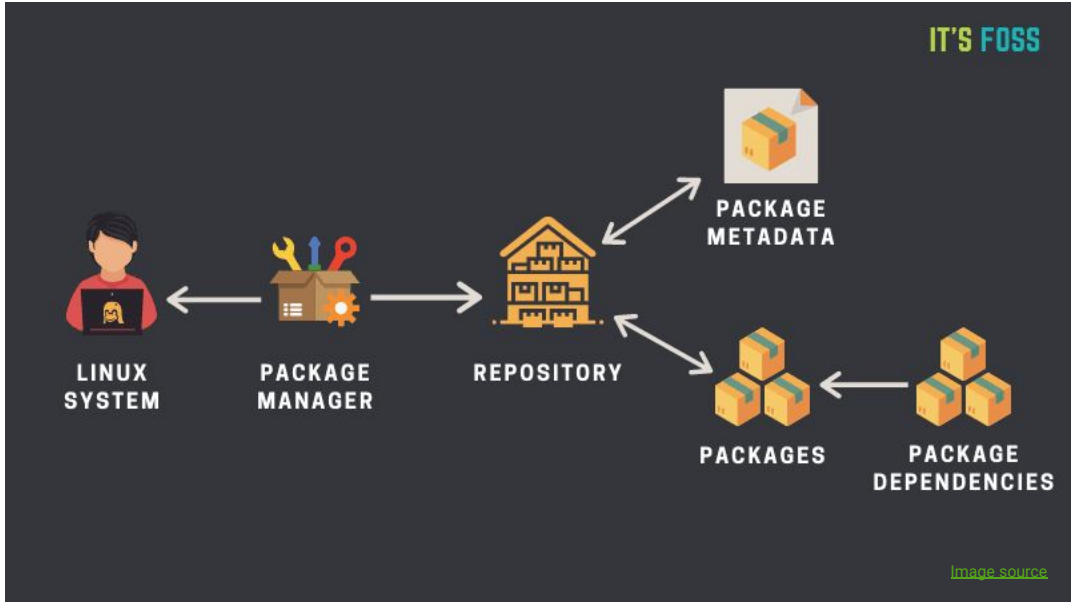
Software dependencies can be very complex

pandas

The reality is quite different…



Package managers help solve these challenges



You may be using package managers already

- Programming Languages
  - R: `install.packages()`, `BiocManager::install()`
  - Python: `pip`, `poetry`, `pipenv`
  - Julia: `Pkg.add()`
- System-level package managers for Linux
  - Debian-based distributions (e.g., Debian, Ubuntu): `apt`
  - Red Hat-based distributions (e.g., Fedora, CentOS, RHEL): `yum`, `rpm`
  - Arch Linux and derivatives (e.g., Manjaro): `pacman`

Local package managers are ideal for bioinformatics

Install and manage software within a user's environment (e.g. your home folder)

No need for administrator or root permissions

→ Ideal for HPC use

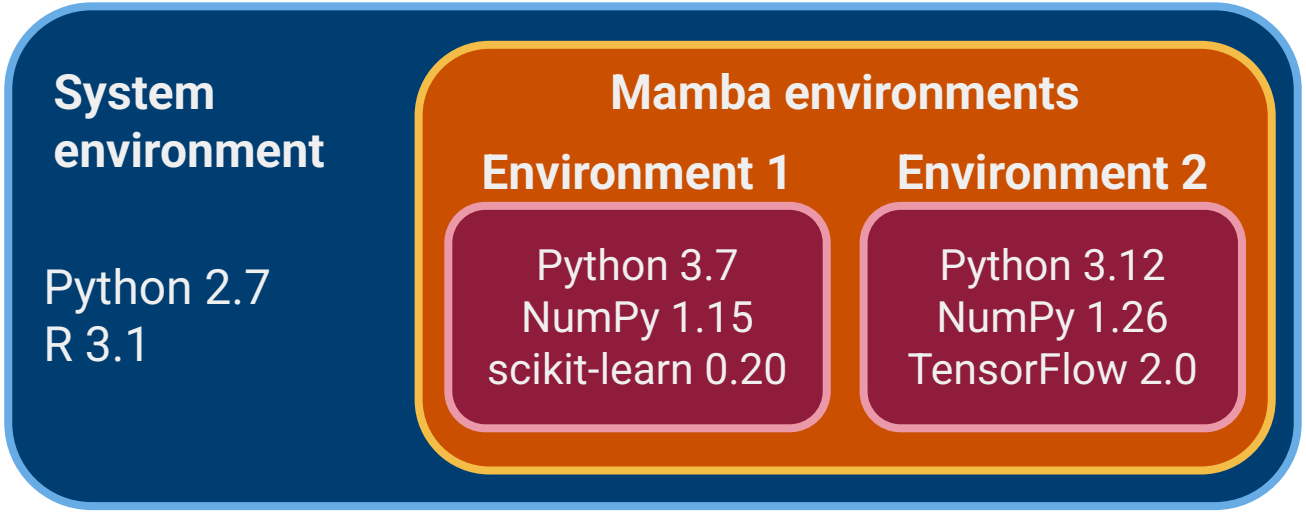
Manage multiple versions of the same software and upgrade them easily

Local package managers are ideal for bioinformatics

Examples:

- **Mamba** (aka Conda) → well adopted by the bioinformatics community
- Homebrew → popular with macOS users, but doesn't support environments
- Pixi → faster alternative to Mamba, but early days
- Spack and Nix → more advanced

Mamba encapsulates software in environments



Environments are effectively separate folders that store separate installations of the software

Mamba installation

If you follow our installation instructions:

- Mamba is installed in your home directory: `~/miniforge3`
- You can test the installation by running

```
mamba activate base
```

before
```
participant@training-pc:~$
```

after
```
(base) participant@training-pc:~$
```

The prompt shows which environment is active

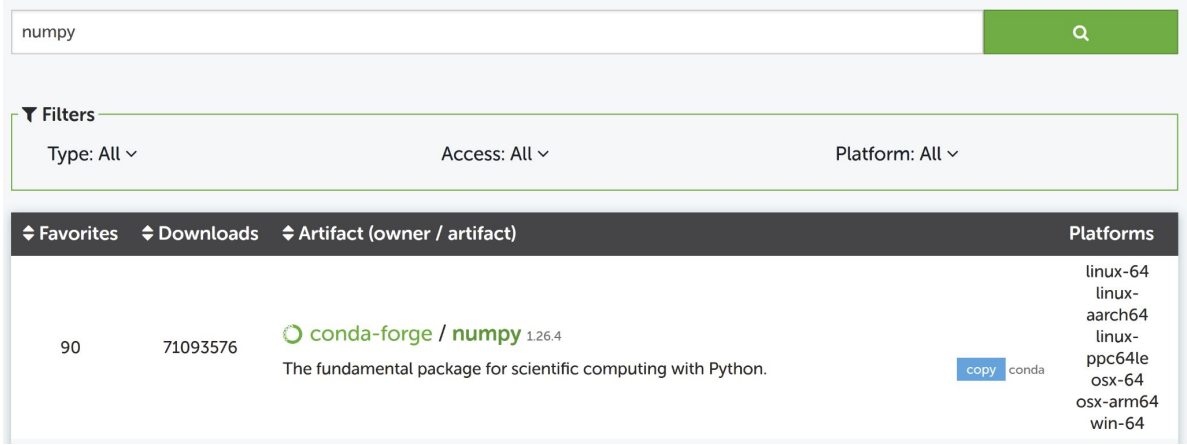
```
(base) participant@training-pc:~$
```

Indicates which environment is currently **active**:

- The default environment is called `(base)`.
- Software installed in the active environment is available to you.
- Software from other environments is not available until you activate them.
- Software installed system-wide remains available as normal.

Step 1: find available packages

Search on anaconda.org



From the terminal

```
mamba search numpy
```

Step 2: create an environment

```
mamba create -n datasci
```

There is no hard rule as to what should constitute an environment. Some advice:

- Avoid generic and/or bloated environments → package version conflicts will emerge and are hard to resolve.
- Create topic-based environments, e.g. `phylo` (phylogenetics software), `rnaseq` (RNA-seq analysis), etc.
- The “safest” is to create a single environment for each package, e.g. `star` (RNA-seq aligner), `iqtree` (phylogenetic trees), etc.

Step 3: Install the packages within the environment

```
mamba install -n datasci -c conda-forge numpy=1.26.4
```

```
mamba install -n datasci -c conda-forge matplotlib=3.8.3
```

`-n` is the environment we want to install the software in

`-c` is the channel where the software is available from (more about channels in a bit)

`=VERSION` specifies specific package versions to install

Step 4: activate the environment to use the software

```
mamba activate datasci
```

Your prompt will change to indicate the new active environment

```
(datasci) participant@training-pc:~$
```

Channels: communities providing installation recipes

A **channel** is a repository used for the distribution of package recipes.

Popular channels:

- `conda-forge` for scientific computing software
- `bioconda` for bioinformatics software

⚠️ Be sure to use these channels for your work and use the *miniforge* installer that we recommend, as there are some licensing issues if using software/channels from Anaconda (the company).

Set default channels during installation

Our Mamba installation instructions include a step to automatically search `conda-forge` and `bioconda`

Therefore, you can omit the channel from the install command. These are all equivalent:

```
mamba install -n datasci -c conda-forge numpy=1.26.4
```

```
mamba install -n datasci conda-forge::numpy=1.26.4
```

```
mamba install -n datasci numpy=1.26.4
```

Use YAML environment files to specify environments

```yaml
name: phylo
channels:
  - conda-forge
  - bioconda
dependencies:
  - iqtree==2.3.3
  - mafft==7.525
```

Environments can be specified in a text-based file in YAML format.

YAML is a text format often used for configuration files in bioinformatics and computer science.

Use YAML environment files to specify environments

```yaml
name: phylo
channels:
  - conda-forge
  - bioconda
dependencies:
  - iqtree==2.3.3
  - mafft==7.525
```

Name of the environment you want to create

Use YAML environment files to specify environments

```yaml
name: phylo
channels:
  - conda-forge
  - bioconda
dependencies:
  - iqtree==2.3.3
  - mafft==7.525
```

Channels that Mamba should search software from in order of priority:

First it will search `conda-forge`, then it will search `bioconda`

(sometimes the same package is available from multiple channels)

Use YAML environment files to specify environments

```yaml
name: phylo
channels:
  - conda-forge
  - bioconda
dependencies:
  - iqtree=2.3.3
  - mafft=7.525
```

The packages you want to install within the environment.

Use YAML environment files to specify environments

```
name: phylo
channels:
  - conda-forge
  - bioconda
dependencies:
  - iqtree=2.3.3
  - mafft=7.525
```

Note on formatting:

- The space between the hyphen and the channel/package is **required**
- The spaces at the beginning of the line are optional (used for eligibility purposes)

Use YAML environment files to specify environments

To install from the file:

```
mamba env create -f phylo.yml
```

Try it!
Go the demo folder and run this command

If you modify the file (e.g. add other packages), you can update the environment:

```
mamba env update -f phylo.yml
```

Why you should use YAML environment files

- Permanent **documentation** of your environment
  - Useful to write methods in your paper
- Ensures **reproducibility**
  - Allows others to recreate your analysis
- Easier to setup in **different compute environments**
  - e.g. same software on your local machine and a HPC server

---

Tip: You can generate a YAML file from an existing environment

```
mamba env export -n phylo > env.yml
```

Disadvantages of Mamba package manager

Package availability

Disk space

Dependency conflicts

Not all packages are available on public channels

E.g. *Cell Ranger*: software for pre-processing single-cell data.

Provided by 10X Genomics with no public Conda recipe.

```
$ mamba search cellranger

No match found for: cellranger. Search: *cellranger*
# Name                       Version           Build  Channel
r-cellranger                     1.1.0 mro343h889e2dd_0  pkgs/r
r-cellranger                     1.1.0 mro350h576908c_0  pkgs/r
r-cellranger                     1.1.0 mro351hf348343_0  pkgs/r
```

Disk space used by Mamba environments can grow

Remove unused environments:

1. List environments and identify which you don't need: `mamba env list`
2. Create YAML before deleting: `mamba env export > myenv.yml`
3. Remove environment: `mamba env remove -n ENV`

Later you can recreate environments from the YAML files:

```
mamba env create -f myenv.yml
```

Occasionally remove cached package installers:

```
mamba clean --all
```

Complex environments lead to dependency conflicts

```yaml
name: metagen
channels:
  - conda-forge
  - bioconda
dependencies:
  - multiqc=1.24.1
  - cutadapt=4.9
  - metaphlan=4.1.1
  - gtdbtk=2.4.0
  - abricate==1.0.1
```

A researcher working in a metagenomics project decided to create a **single environment** for all the software they needed.

Complex environments lead to dependency conflicts

After a long time where Mamba tried to resolve dependency graph, a very long message was printed…

```
The following packages are incompatible
├─ abricate 1.0.1  is not installable because it requires
│  ├─ perl-bioperl >=1.7  but there are no viable options
│  │  ├─ perl-bioperl [1.7.2|1.7.8] would require
│  │  │  ├─ perl >=5.26.2,<5.26.3.0a0 , which can be installed;
│  │  │  ├─ perl-bio-samtools with the potential options
│  │  │  │  ├─ perl-bio-samtools 1.43 would require
│  │  │  │  │  └─ perl >=5.22.0,<5.23.0 , which conflicts with any installable
│  │  │  │  ├─ perl-bio-samtools 1.43 would require
│  │  │  │  │  ├─ libzlib >=1.2.13,<1.3.0a0  with the potential options
│  │  │  │  │  │  ├─ libzlib 1.2.13 would require
│  │  │  │  │  │  │  └─ zlib 1.2.13 *_6, which can be installed;
│  │  │  │  │  │  ├─ libzlib 1.2.13 would require
│  │  │  │  │  │  │  └─ zlib 1.2.13 *_4, which can be installed;
│  │  │  │  │  │  └─ libzlib 1.2.13 would require
│  │  │  │  │  │     └─ zlib 1.2.13 *_5, which can be installed;
│  │  │  │  │  ├─ perl >=5.32.1,<5.33.0a0 *_perl5, which conflicts with any ins
```

Solution
break it into smaller environments
(as we said earlier)

Exercises

Put these new skills in practice in the exercises found in the materials

1. Create a new environment using a YAML file.
2. Update an existing environment.
3. Explore issues of package dependency conflicts.

Activating an environment non-interactively

When submitting jobs to SLURM on the HPC you need to include the following lines of code:

```
# Always add these two commands to your scripts
eval "$(conda shell.bash hook)"
source $CONDA_PREFIX/etc/profile.d/mamba.sh

# then you can activate the environment
mamba activate datasci
```

Mamba tips summarised

Keep a note of what you installed in each environment

- Make use of `yml` environment files

You may be tempted to install everything on `(base)`

- Resist that temptation! Sooner or later your environment will have dependency conflicts and you will “break” your Mamba

Over time Mamba can take a lot of disk space:

- Remove environments you don’t need regularly
- Run **`mamba clean`** to remove unused and cached packages

Summary: package managers

Easy way to manage software locally (HPC-friendly).

Automatically installs dependencies.

Encapsulate different software versions in environments.

Recreate environments on other computers easily (see *Conda* docs)

Packages are not optimally compiled for the hardware in use.

For complex environments dependency conflicts can be hard (or impossible) to resolve.

Not every software available

Can take a lot of space

Software containers

How to use Singularity to run bioinformatics software

Software containerisation

Encapsulates software and dependencies in isolated, reproducible environments

- Consistency across different computational setups
- Simplifies deployment of complex bioinformatics pipelines
- Facilitates sharing and collaboration
- Reduces conflicts between software versions and dependencies
- Commonly implemented using Docker or **Singularity**

Singularity containers for bioinformatics

- You can build your own containers from Dockerfiles
  (out of the scope of this course, but check this one if you're interested)
- Bioinformatics community has built containers for many common software packages → ready to be used by anyone!
  - BioContainers → great first place to look for bioinformatics images. Includes results from:
    - Galaxy depot → pre-built containers (SIF files)
    - DockerHub
    - Quay.io
  - Sylabs → generic repo, but not always the most up-to-date for bioinformatics

Using Singularity images

1. Pull the image

```
singularity pull image-name.sif link-to-image
```

2. Run the command in the container

```
singularity run image-name.sif command to run in the container
```

Filesystem binding

By default only some directories are available within the Singularity container (see the docs to find out which ones)

What if you want to use data from a non-standard directory?

```
singularity run example.sif \
  mapper --ref /databases/ref/human --in  reads/example.fq
```

```
Error: /databases/ref/human no such file or directory
```

Filesystem binding

By default only some directories are available within the Singularity container (see the docs to find out which ones)

What if you want to use data from a non-standard directory?

```
singularity run example.sif --bind /databases/ref/ \
  mapper --ref /databases/ref/human --in  reads/example.fq
```

✅ Works!

Exercises

Put these new skills in practice in the exercise found in the materials

- Run a command within a singularity image

Summary: software containers

- Encapsulates an entire operating system, software and dependencies in a portable container (image file).
- Running complex software environments in a reproducible and portable way.
- Independent of the host system, ensuring full reproducibility.
- Containers can be moved between different machines with no compatibility issues.
- Many bioinformatics packages available as pre-built images.
    - Otherwise building container images can be challenging.

Workflow management systems

How to use Nextflow to run complex bioinformatics pipelines

Learning objectives

- Describe at a high-level what a workflow/pipeline is.
- Recognise advantages and disadvantages of using standardised pipelines.
- Access the wide range of curated pipelines available from the nf-core community.
- Use a simple pipeline and prepare its required input sample sheet.
- Learn how to write configuration files to suit your computational needs.
- Apply these skills to a “real-world” dataset using nf-core pipelines.

Workflows, pipelines and processes: terminology

The terms **workflow and pipeline are used interchangeably**

- **Workflow**: Overall structure of an analysis, which may have multiple entry points and not be linear.

  An RNA-seq workflow, with multiple options for quantifying gene expression.

- **Pipeline**: A series of linear steps; the output from one step feeds to the next.

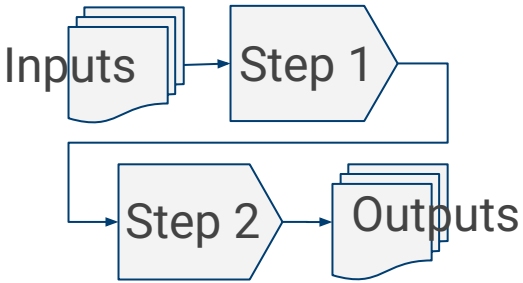
  FASTQ → FASTP → SALMON

- **Process / rule**: a specific step in the pipeline.

  Quality-trimming reads with `fastp`

Automated workflows are ideal for bioinformatics

Workflow	Previous approach	Current approach
Sequences of related tasks	Ad hoc Bash/Python/Perl scripts	Dedicated workflow management systems



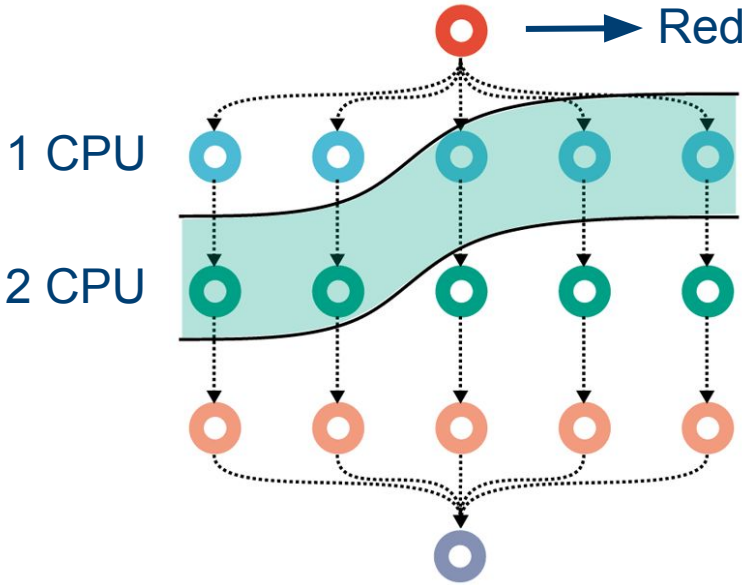
Poor scaling

nextflow

snakemake

Workflow managers use optimal scheduling of tasks

Highlighted tasks are ready to run.
We have 5 CPUs available.



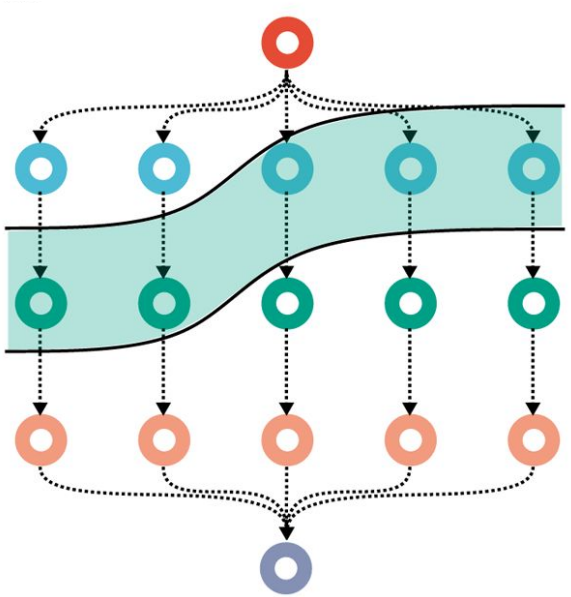
Red task produces a large intermediate file that can be removed once blue completes

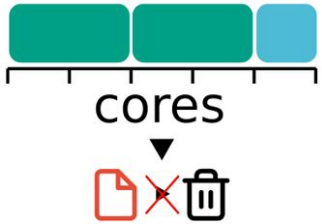
How should we schedule them?

A) 2 + 1

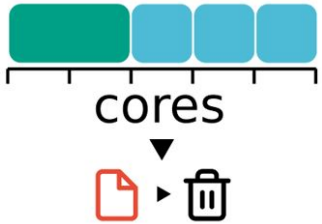
B) 1 + 3

Workflow managers use optimal scheduling of tasks





X



✔

Second option is best as we can remove the large temporary file more quickly

Workflow management systems: advantages

- **Reproducible & reusable** - consistency across runs.
- **Scalable & efficient** - optimised for HPC and cloud environments.
- **Fault-tolerant** - supports “checkpoint and resume” of failed tasks.
- **Automation** - manages dependencies and software installation.
- **Flexible & customisable** - options to suit specific analysis needs.
- **Reduces human error** – uses best practices and standardisation.
- **Long-term maintainability** - through collaboration and versioning.

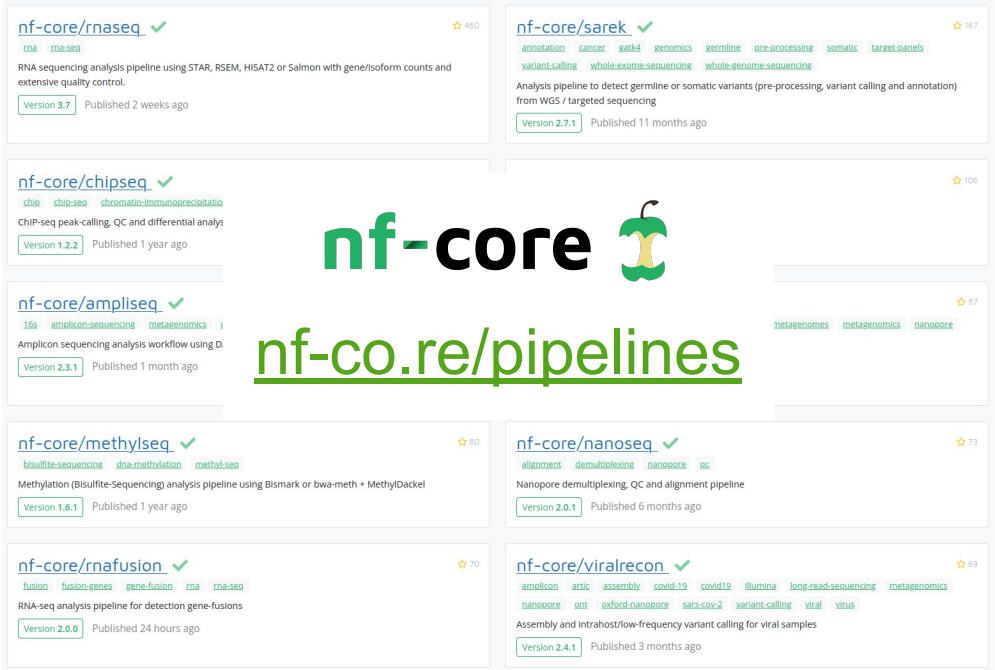
Workflow management systems: disadvantages

- **Bioinformatic expertise** - requires enough knowledge to make the right choices for the analysis task at hand.
- **Complex setup** - configuration can be time-consuming.
- **Overhead for simple tasks** - may be excessive for small-scale analyses.
- **Limited control over updates** - public workflows may lag behind in software versions or lack options the user may want.
- **Developer burden** - requires rigorous documentation, testing & review.

nf-core provides a wide range of high-quality pipelines

Nextflow Core Team:

- Development of high-quality pipelines
- Regularly tested
- Excellent documentation
- Many many applications



Nextflow increased in popularity due to nf-core

Nextflow has become one of the most popular WFMS, in part thanks to its very active community nf-core.

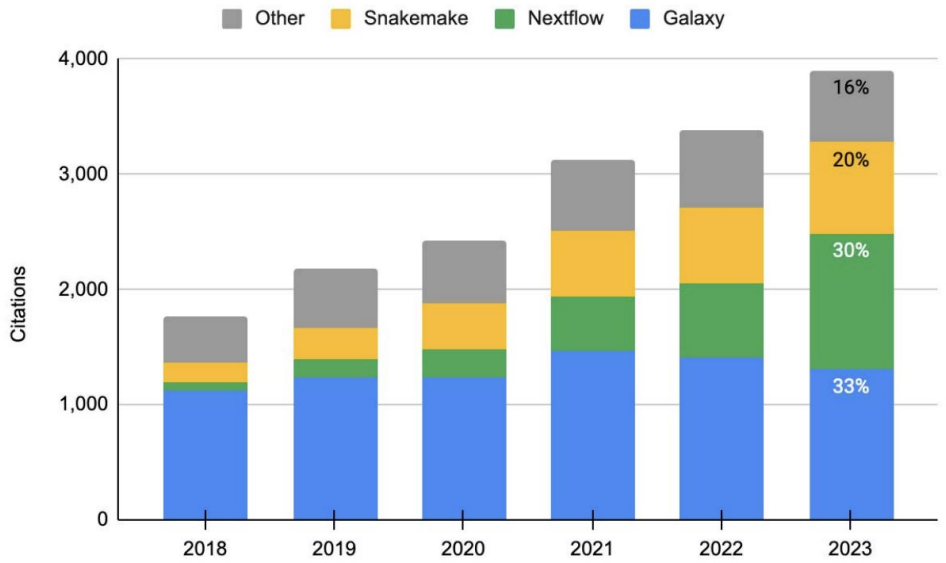
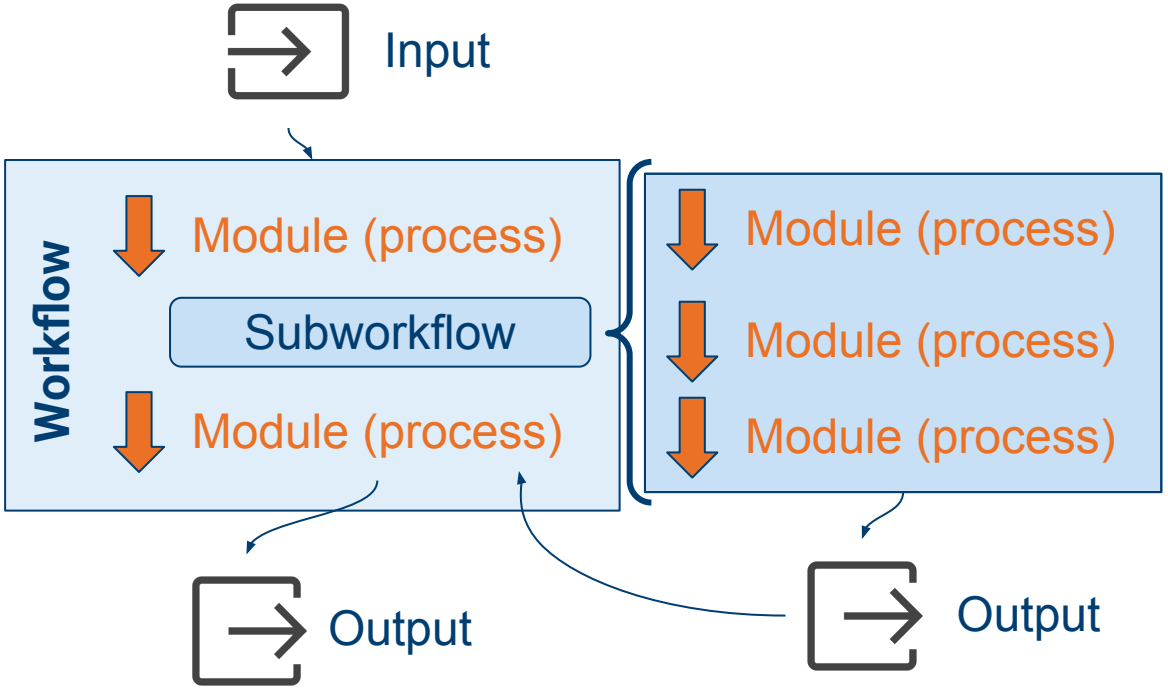


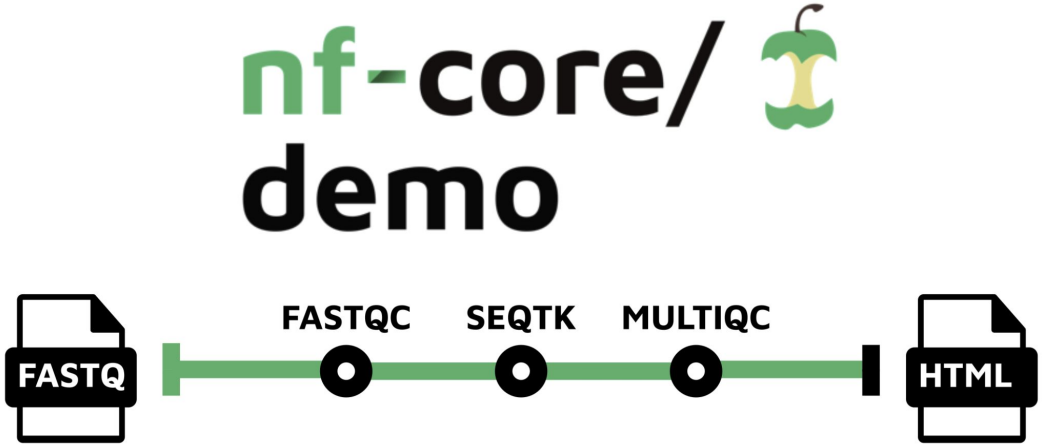
FIGURE 1: Google Scholar citation counts for bioinformatics workflow management systems. Sum of citations of the major publications of Galaxy, Nextflow, and Snakemake between 2018 and 2023 (Data in Supplementary Table 1).

https://doi.org/10.1101/2024.05.10.592912

General nf-core workflow diagram



Example of a very simple workflow



Use the ‘nextflow run’ command to run the pipeline

```
nextflow run nf-core/demo \
  -profile "singularity" \
  -revision "1.0.1"  \
  -resume \
  --input "samplesheet.csv" \
  --outdir "results/qc" \
  --fasta "my_genome.fa.gz"
```

Use the ‘nextflow run’ command to run the pipeline

```
nextflow run nf-core/demo \
  -profile "singularity" \
  -revision "1.0.1"  \
  -resume \
  --input "samplesheet.csv" \
  --outdir "results/qc" \
  --fasta "my_genome.fa.gz"
```

The workflow we want to run.

Any workflow hosted on github can be used.

For example, this pipeline can be run with:

```
nextflow run avantonder/assembleBAC
```

Use the ‘nextflow run’ command to run the pipeline

```
nextflow run nf-core/demo \
  -profile "singularity" \
  -revision "1.0.1"  \
  -resume \
  --input "samplesheet.csv" \
  --outdir "results/qc" \
  --fasta "my_genome.fa.gz"
```

Indicates we want to use Singularity to manage the software (recommended).

conda is also a valid option but is less stable.

Use the ‘nextflow run’ command to run the pipeline

```
nextflow run nf-core/demo \
  -profile "singularity" \
  -revision "1.0.1"  \
  -resume \
  --input "samplesheet.csv" \
  --outdir "results/qc" \
  --fasta "my_genome.fa.gz"
```

The version of the workflow we want to use.

It is recommended to include this, as workflows often get updated, so this ensures reproducibility and consistency across your analysis.

Use the ‘nextflow run’ command to run the pipeline

```
nextflow run nf-core/demo \
  -profile "singularity" \
  -revision "1.0.1"  \
  -resume \
  --input "samplesheet.csv" \
  --outdir "results/qc" \
  --fasta "my_genome.fa.gz"
```

In case of a previous failed run, Nextflow will restart from where it previously failed.

Without this option, Nextflow will always start from the beginning.

Use the ‘nextflow run’ command to run the pipeline

```
nextflow run nf-core/demo \
  -profile "singularity" \
  -revision "1.0.1"  \
  -resume \
  --input "samplesheet.csv" \
  --outdir "results/qc" \
  --fasta "my_genome.fa.gz"
```

Usually a CSV file specifying sample names and respective input files.

(more about this later)

Use the ‘nextflow run’ command to run the pipeline

```
nextflow run nf-core/demo \
  -profile "singularity" \
  -revision "1.0.1"  \
  -resume \
  --input "samplesheet.csv" \
  --outdir "results/qc" \
  --fasta "my_genome.fa.gz"
```

Output directory to save the results.

Recommended to use a separate directory from your raw data files to avoid accidental overwrites.

Use the ‘nextflow run’ command to run the pipeline

```
nextflow run nf-core/demo \
  -profile "singularity" \
  -revision "1.0.1"  \
  -resume \
  --input "samplesheet.csv" \
  --outdir "results/qc" \
  --fasta "my_genome.fa.gz"
```

The reference genome to use for the analysis (when relevant, depending on the pipeline).

Don’t use igenomes as they are not up-to-date.

Remember our comments about reference genomes a few slides ago

General Nextflow options use single hyphen

- `-profile` → configuration profile to use
  - These are available by default: `singularity`, `docker`, `conda`
  - Further configurations possible (more on this later)
- `-revision` / `-r` → version of the pipeline to use.
- `-resume` → restart the pipeline where it last stopped (uses cached results).
- `-work-dir` → directory where intermediate result files are stored.
  - Default is a directory called '**work**' in the folder where you run nextflow run command.

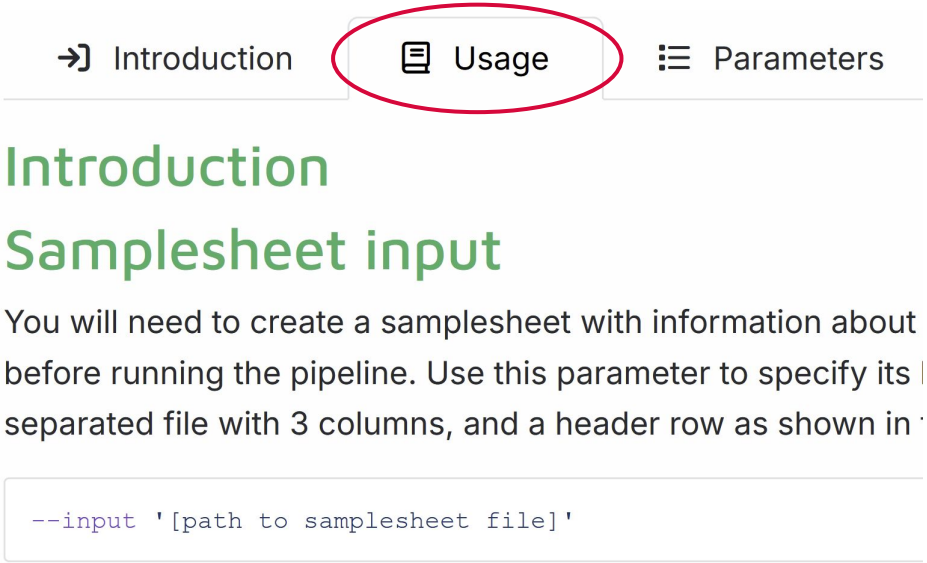
Pipeline-specific options use double hyphen

Common options in many **nf-core** pipelines:

- `--input` → usually a CSV file specifying sample names and respective input files.
- `--outdir` → the output directory to save the results.
- `--fasta` → reference genome.
- `--gff` / `--gtf` → reference gene annotation.

The input sample sheet

Format for the samplesheet is detailed in the pipeline **documentation**



Typical format of input samplesheet

- Sample name
- Path to FASTQ file (read 1)
- Path to FASTQ file (read 2)
- **Extra columns** depending on the pipeline

```
sample,fastq_1,fastq_2
CONTROL_REP1,AEG588A1_R1.fastq.gz,AEG588A1_R2.fastq.gz
CONTROL_REP2,AEG588A2_R1.fastq.gz,AEG588A2_R2.fastq.gz
CONTROL_REP3,AEG588A3_R1.fastq.gz,AEG588A3_R2.fastq.gz
```

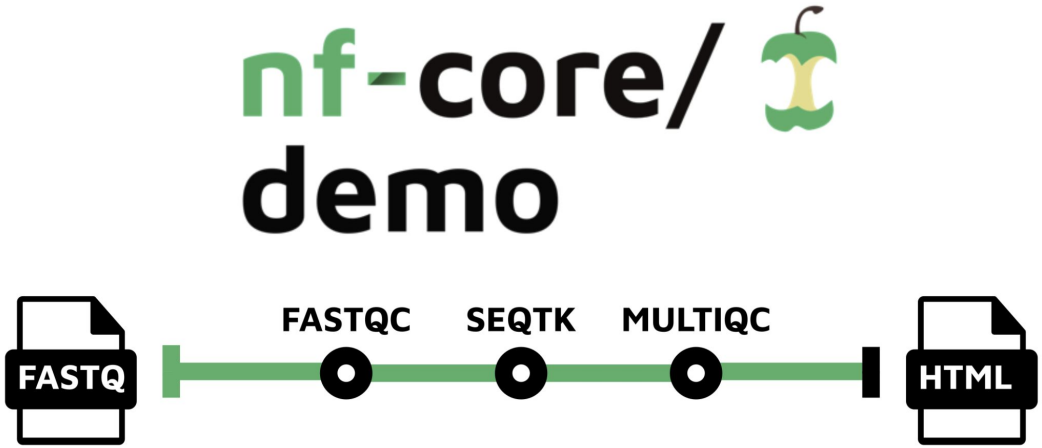
The input samplesheet is flexible

- If you re-sequenced a sample, pipeline will merge the data based on the sample name
- A mixture of single-end and paired-end reads is often supported

```
sample,fastq_1,fastq_2
CONTROL_REP1,AEG588A1_R1.fastq.gz,AEG588A1_R2.fastq.gz
CONTROL_REP2,AEG588A2_R1.fastq.gz,
CONTROL_REP3,AEG588A3_R1.fastq.gz,AEG588A3_R2.fastq.gz
CONTROL_REP3,AEG598A4_R1.fastq.gz,AEG598A4_R2.fastq.gz
```

Hands-on time: nf-core/demo

Course materials folder: `demo`
Script with nextflow command: `scripts/02-run_nfcore_demo.sh`



Troubleshooting

```
Execution cancelled -- Finishing pending tasks before exit
-[nf-core/demo] Pipeline completed with errors-
ERROR ~ Error executing process > 'NFCORE_DEMO:DEMO:FASTP (drug_rep1)'

Caused by:
  Process requirement exceeds available memory -- req: 36 GB; avail: 23.5 GB

Command executed:

  [ ! -f  drug_rep1_1.fastq.gz ] && ln -sf SRR7657874_1.downsampled.fastq.gz drug_rep1_1.fastq.gz
  [ ! -f  drug_rep1_2.fastq.gz ] && ln -sf SRR7657874_2.downsampled.fastq.gz drug_rep1_2.fastq.gz
  fastp \
      --in1 drug_rep1_1.fastq.gz \
      --in2 drug_rep1_2.fastq.gz \
      --out1 drug_rep1_1.fastp.fastq.gz \
      --out2 drug_rep1_2.fastp.fastq.gz \
      --json drug_rep1.fastp.json \
      --html drug_rep1.fastp.html \
       \
       \
       \
      --thread 6 \
      --detect_adapter_for_pe \
       \
      2> >(tee drug_rep1.fastp.log >&2)

  cat <<-END_VERSIONS > versions.yml
  "NFCORE_DEMO:DEMO:FASTP":
      fastp: $(fastp --version 2>&1 | sed -e "s/fastp //g")
  END_VERSIONS
```

Don’t be put off by long error messages.

Read it carefully, usually it’s only a small part of the message that is relevant.

Can you see what the problem was in this case?

Exercises

Put these new skills in practice in the exercises found in the materials

1. Prepare a samplesheet for a pipeline of your choice.
2. Run the pipeline.

Note: In exercise 2, some of the pipelines take a long time to run! Don’t stare at the computer waiting for them to fully finish - consider the exercise complete once you launch the pipeline successfully and revisit the results later.

Summary: workflow management systems

- **Automate complex pipelines**, managing inputs, outputs, resource allocation and software installation.
- Popular workflow managers include **Snakemake** and **Nextflow**.
- The **nf-core** community maintains a large collection of highly-curated bioinformatic pipelines.
- Most nf-core pipelines require an **input samplesheet** (CSV file).
- Pipelines are **highly configurable** using pipeline-specific options.

} Read the docs!

Nextflow on the HPC

How to configure Nextflow to run on a HPC cluster

Running jobs on a HPC

On a HPC we typically use a **Job Scheduler** to submit jobs:

SLURM, LSF, PBS, HTCondor, …

Nextflow has the ability to **orchestrate job submission** using any one of these schedulers (and many more)

- The components where the jobs are run are called **executors**
- We define executor properties using a **configuration file**

Nextflow HPC configuration: process definition

```
process {
  executor = 'slurm'
  queue = 'cclake'
  resourceLimits = [
    cpus: 56,
    memory: 192.GB,
    time: 36.h
  ]
}
```

Example:
config file for the Cascade Lake Nodes at Cambridge

Instruct Nextflow that:

- The pipeline processes (tasks) will be run using **SLURM** as the executor.
- They should run in the queue called **cclake**.
- The maximum number of **CPUs**, **RAM memory** and **time limit** allowed in the chosen queue.

Nextflow HPC configuration: executor definition

```
executor {
    account              = 'CASTLE-SL2-CPU'
    perCpuMemAllocation  = '3410MB'
    queueSize            = '200'
    pollInterval         = '3 min'
    queueStatInterval    = '5 min'
    submitRateLimit      = '50sec'
    exitReadTimeout      = '5 min'
}
```

Options to use when submitting the jobs. These are often equivalent to existing options in the scheduler you normally use.
See the docs for more information.

Nextflow HPC configuration: singularity definition

```
singularity {
    enabled = true
    autoMounts = true
    pullTimeout = '1 h'
    cacheDir = 'PATH/TO/nextflow-singularity-cache'
}
```

Not specific to HPC, but a good idea to set a **cache directory** to save Singularity images, so you don't need to download them again if you run the same pipeline again.

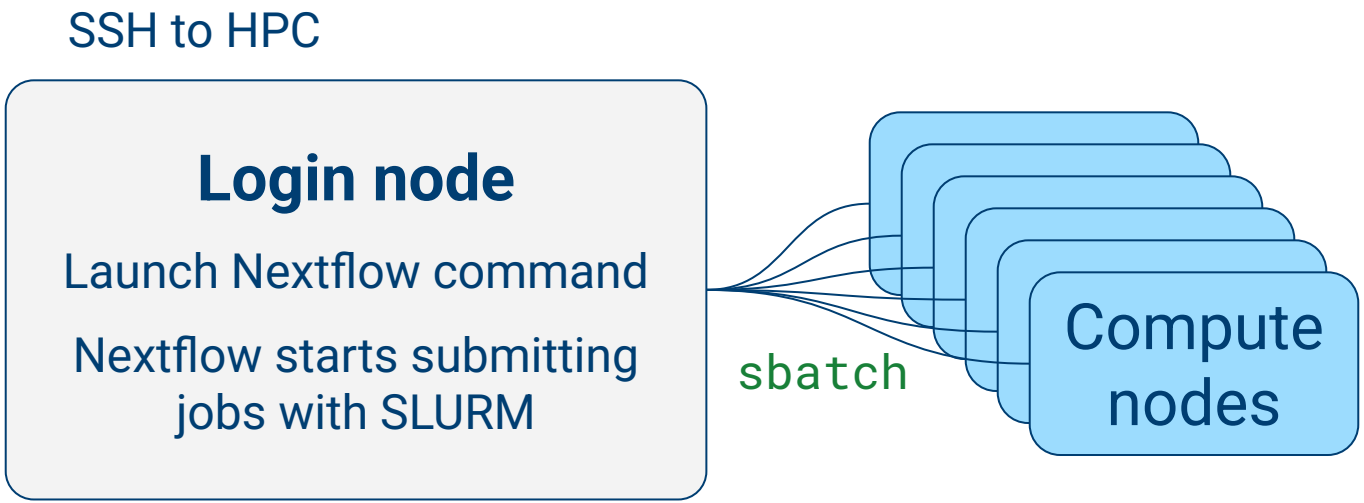
Nextflow HPC configuration: all together

myhpc.config

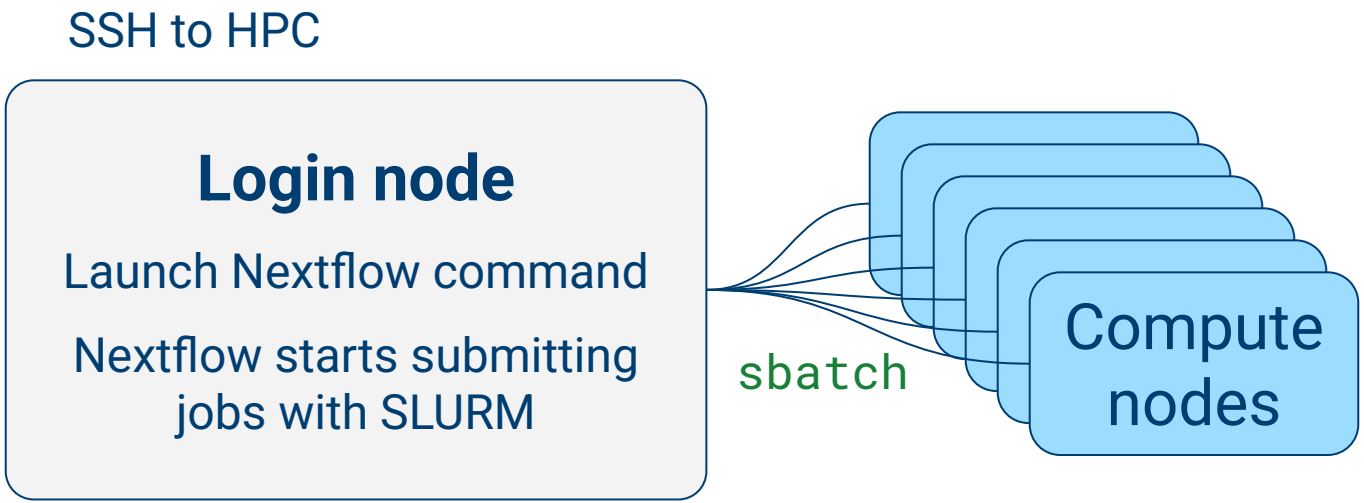
```
process {
    executor = 'slurm'
    queue = 'cclake'
}
executor {
    account             = 'YOUR-BILLING-ACCOUNT'
    perCpuMemAllocation = '3410MB'
    queueSize           = '200'
    pollInterval        = '3 min'
    queueStatInterval   = '5 min'
    submitRateLimit     = '50sec'
    exitReadTimeout     = '5 min'
}
params {
          max_memory = '192.GB'
          max_cpus = '56'
          max_time = '12.h'
}
singularity {
    enabled = true
    autoMounts = true
    pullTimeout = '1 h'
    cacheDir = 'PATH/TO/nextflow-singularity-cache'
}
```

```
nextflow run <some-workflow> \
  -c "myhpc.config" \
  -profile "singularity" \
  --etc... pipeline options
```

Keep Nextflow running after logout



Keep Nextflow running after logout



How can I logout without killing the Nextflow process?

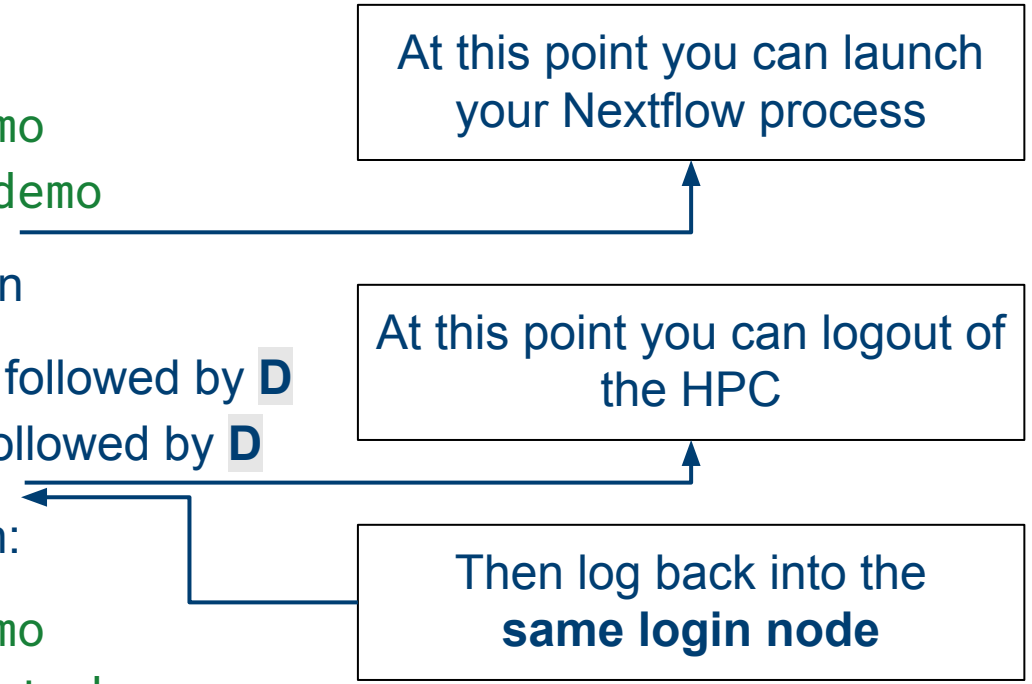
Use terminal multiplexer to run your nextflow process

- Usually used to have multiple terminals on the same window
- They have the ability to “detach” a session, which stays running in the background
    - Great feature to use with a process we want to leave running on a login node of the HPC

- Two main multiplexers (usually available on standard Linux):
    - `screen`
    - `tmux`

How to use a terminal multiplexer

- Start a session
  - `screen -S demo`
  - `tmux new -s demo`
- Detach from a session
  - Screen: **Ctrl + A** followed by **D**
  - Tmux: **Ctrl + B** followed by **D**
- Reattach to a session:
  - `screen -r demo`
  - `tmux attach -t demo`



Exercises

Put these new skills in practice in the exercises found in the materials

- Login to our training HPC
- Create a configuration file
- Edit Nextflow command to use the configuration file
- Start a `tmux`/`screen` session
- Launch the pipeline
- Detach the session
- Check the queue
- (optional) Logout, log back in, reattach your terminal and confirm that Nextflow is still happily running

Using tmux/screen on large HPC servers

At Cambridge HPC

```
ssh user@login.hpc.cam.ac.uk
```

assigns you to one of many login nodes, e.g.:

`login-p-1` , `login-p-2` , `login-p-3` , `login-p-4`

To retrieve your `tmux`/`screen` session you need to

log back in to the same node, e.g.

```
ssh user@login-q-3.hpc.cam.ac.uk
```

Summary: nextflow on the HPC

Nextflow can automatically submit and manage jobs with a wide range of job schedulers on HPC systems.

- Use a configuration file to specify job submission options specific to your server and account type.
- Make sure to cache the singularity images to save bandwidth and time re-running workflows.
- Use the config file with the `-c` option.
- Use a terminal multiplexer (`tmux`/`screen`) to have a persistent terminal that can be retrieved after logout.

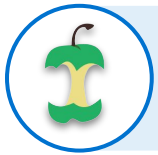
Reproducible and scalable bioinformatics

Bringing it all together

Tools to enhance your bioinformatics projects

Manage your local software using Mamba

Use software containers as an alternative to Mamba



Use curated nf-core pipelines for your bioinformatic analyses

Most of the time you don't use these directly, but Nextflow will

Configure Nextflow to run on a HPC to scale your analyses

Worked example: setting up a new RNA-seq analysis

Step 1: high-level outline

- Pre-processing using `nf-core/rnaseq` pipeline
  - I will need Nextflow installed on the HPC.
  - I need reference genome for my species.
- Downstream analysis in R using `DESeq2`
  - I will use my local R + RStudio installation.
  - I also want to run some of the analysis on the HPC, so will need to install `DESeq2` in there.

Worked example: setting up a new RNA-seq analysis

Step 2: prepare your directories

- As a rule of thumb keep separate directories for:
  - Raw data
  - Public data (genomes, etc.)
  - Results
  - Scripts

```
myproject
   |_ reads
   |_ results
   |_ reference
   |_ scripts
         |_ shell
         |_ R
```

Worked example: setting up a new RNA-seq analysis

Step 3: download public data

- My project is on Drosophila, and I'll use Ensembl reference

```
# work from reference directory
cd reference
# download genome
wget "link_to_drosophila_genome"
# download annotation
wget "link_to_GFF_file"
```

Worked example: setting up a new RNA-seq analysis

Step 4: setup my software using Mamba (on the HPC)

```
mamba create -n nextflow nextflow=24.10.5


mamba create -n deseq bioconductor-deseq2=1.46.0
```

Worked example: setting up a new RNA-seq analysis

Step 5: write a Nextflow configuration file for the HPC

```
process {
    executor = 'slurm'
    queue = 'cclake'
}

...etc...
```

Set your bioinformatics project up for success

1. High-level outline
2. Prepare project directories
3. Download public data
4. Setup software on the HPC with **Mamba**
5. Write **Nextflow** configuration files

🚀 Ready for the analysis!

(now spend lots of time reading pipeline/software documentation…)

Please give us your feedback

Certificate of attendance: Link is provided at the end of the feedback survey 🥕